Request for Proposal (RFP)

Frontend UI & Product Design — AI-Powered RAG Platform

Issued by: LaunchLive Studio
Document Version: 1.0
Date: May 1, 2026
Project Code: LLS-RAG-FE-001

---

1. Overview

LaunchLive Studio is developing an AI-powered Retrieval-Augmented Generation (RAG) platform that allows businesses to ingest their documents and query them using natural language. We are seeking a skilled frontend designer/developer to build a complete, production-ready product UI — including a public-facing marketing landing page and a full internal application dashboard.

The goal is a polished, modern SaaS product experience — comparable in quality and feel to tools like Notion AI, Perplexity, or Mintlify.

---

2. Project Scope

The project consists of two distinct deliverables:

2.1 Public Landing Page (Marketing Site)

A high-converting, visually stunning landing page that presents the RAG platform as a premium AI product.

Required Sections:

#	Section	Description
1	Hero	Bold headline, product tagline, animated or illustrated visual, two primary CTAs: **"Get Trial"** and **"Join Waitlist"**
2	How It Works	3-step visual flow: Upload Docs → Ask Questions → Get Answers with Sources
3	Features	Highlight: multi-format ingestion, semantic search, source citations, conversation history
4	Use Cases	Cards showing target industries: Legal, HR, Finance, Education, Healthcare
5	Testimonials / Social Proof	Placeholder-ready testimonial cards
6	Pricing Teaser	Simple "Early Access" pricing CTA to capture leads
7	FAQ	Accordion-style, minimum 6 questions
8	Footer	Logo, links, social handles, copyright

CTA Behavior:

- **"Get Trial"** → Opens a modal or redirects to a trial signup form (email + name)
- **"Join Waitlist"** → Opens a waitlist modal with email capture + optional message field

Both CTAs must be prominently featured in the Hero and optionally repeated in at least one more section.

2.2 Application Dashboard (Logged-In UI)

A full internal product interface with the following views/pages:

A. Ingest Panel — `/dashboard/ingest`

Tabbed interface with three modes:

- **File Upload Tab —** Drag-and-drop zone accepting PDF, DOCX, TXT, MD, CSV. Show file name, size, and upload progress bar.
- **URL Tab —** Input fields for URL and optional Label. Submit button.

- **Text Tab** — Large textarea for raw text, Title field, optional Metadata (key-value pairs UI). Submit button.

Common elements: single "Ingest" button, success/error toast notifications, loading state.

B. Document Library — `/dashboard/documents`

- Table/card list of all ingested documents
- Columns: Document Name, Type, Status (badge: Processing / Ready / Failed), Chunk Count, Date Added
- Per-row actions: View Details, Delete (with confirmation dialog)
- Document detail drawer/modal showing chunk count and metadata
- Empty state with a prompt to ingest the first document

C. Chat Interface — `/dashboard/chat`

- Full-page chat layout (sidebar + main area)
- Message bubbles (user right, AI left)
- Each AI response includes collapsible **Sources** panel listing referenced chunks
- Conversation ID managed in state
- "New Chat" button in sidebar
- Conversation history list in sidebar (click to reload)
- "Clear Conversation" button in chat header

D. Semantic Search — `/dashboard/search`

- Prominent search bar at top
- Results displayed as source chunk cards — each card shows: source document name, chunk text excerpt, relevance indicator
- No LLM answer, just raw semantic matches
- Empty/no-results state

E. Conversation History — `/dashboard/history`

- List of all past conversations
- Each entry: conversation ID (friendly name or timestamp), message count, last active
- Click to open in Chat view
- Delete individual conversations with confirmation

3. Design Requirements

3.1 Visual Direction

- **Style:** Modern, minimal-yet-bold SaaS product (reference: Linear, Perplexity, Anthropic Console)
- **Theme:** Light mode primary, dark mode support preferred
- **Feel:** Professional, trustworthy, AI-native — not generic template-style

3.2 Typography

- Distinctive display font for headings (e.g., Sora, Clash Display, Cal Sans — no Inter/Roboto/Arial)
- Clean, readable body font
- Strong typographic hierarchy

3.3 Color Palette

- Propose a cohesive brand palette
- Primary accent color used for CTAs and highlights
- Neutral grays for content areas
- Semantic colors for status badges (green = Ready, amber = Processing, red = Failed)

3.4 Animations & Interactions

- Subtle page-load animations on the landing page
- Smooth tab transitions in Ingest Panel
- Toast notifications for all async actions (upload, delete, ingest success/failure)
- Hover states on all interactive elements
- Skeleton loaders for data-fetching states

3.5 Responsiveness

- Fully responsive across desktop, tablet, and mobile
- Landing page must be pixel-perfect on mobile
- Dashboard: responsive with collapsible sidebar on smaller screens

4. Technical Requirements

4.1 Stack (Preferred)

- **Framework:** Next.js 14+ (App Router)
- **Styling:** Tailwind CSS
- **Animations:** Framer Motion
- **Component Library:** Shadcn/UI or Radix UI primitives
- **Icons:** Lucide React

4.2 API Integration

The frontend must integrate with the following REST API endpoints provided by the backend team:

Ingest Routes (`/api/ingest`)

Method	Endpoint	Usage
POST	`/api/ingest/file`	File upload (multipart/form-data)
POST	`/api/ingest/url`	Ingest from URL
POST	`/api/ingest/text`	Ingest raw text
DELETE	`/api/ingest/{doc_id}`	Delete a document
GET	`/api/ingest/documents`	List all documents
GET	`/api/ingest/documents/{doc_id}`	Get single document status

Query Routes (`/api/query`)

Method	Endpoint	Usage
POST	`/api/query/ask`	Main Q&A — returns answer + sources
POST	`/api/query/search`	Semantic search — returns top-k chunks
GET	`/api/query/conversations/{id}`	Get conversation history
DELETE	`/api/query/conversations/{id}`	Clear conversation

Base URL and auth headers will be provided separately. All API calls should include proper error handling and loading states.

4.3 State Management

- Use React Context or Zustand for global state (conversation ID, document list)
- No Redux unless strongly justified

4.4 Code Quality

- TypeScript required
- ESLint + Prettier configured
- Folder structure following Next.js App Router conventions
- Reusable component library (`/components/ui`)
- API layer abstracted into service files (`/lib/api`)

---

5. Deliverables

#	Deliverable	Format
1	Figma / Design File	`.fig` or shared Figma link
2	Landing Page (production-ready)	Next.js page + components
3	Dashboard — all 5 views	Next.js pages + components
4	API service layer	TypeScript files
5	README with setup instructions	Markdown
6	Source code	GitHub private repo access

---

6. Timeline

Milestone	Deliverable	Target
M1	Design mockups (Landing + Dashboard)	Week 1